Lesson 4 Notes: Complete C++ Diagonal Matrix Program

1. Main idea of Lesson 4

Lesson 4 continues the same diagonal matrix idea from Lessons 1, 2, and 3.

We are **not learning a new matrix type yet**.

We are still working with a **diagonal matrix**.

A diagonal matrix stores meaningful values only on the main diagonal.

Example:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

The important values are:

```
5, 8, 9, 12
```

So the compact stored array is:

```
A = [5, 8, 9, 12]
```

The diagonal matrix rule is still:

```
if (i == j)
```

That means:

- If `i == j`, the position is on the main diagonal.
- If `i != j`, the position is outside the main diagonal.
- Outside diagonal positions are structural zeros.
- Structural zeros are not stored.

The index formula for diagonal positions is still:

```
A[i - 1]
```

Lesson 4 focuses on writing a **complete C++ diagonal matrix class program**.

The main new ideas are:

- Default constructor
- Parameterized constructor
- Destructor
- `Set()` method
- `Get()` method
- `Display()` method
- 1-based matrix positions in `Set()` and `Get()`
- 0-based loops in `Display()`
- Testing the class in `main()`
- Tracing the program output
- Understanding time and space complexity

---

2. What you already know before Lesson 4

Before Lesson 4, you should already understand these ideas:

Concept	Meaning
Diagonal matrix	A matrix where only the main diagonal values are stored.
Main diagonal	Positions where row number and column number are the same.
Diagonal condition	`i == j`
Non-diagonal position	A position where `i != j`
Structural zero	A zero that exists because of the matrix rule, not because it is stored.
Compact array	A 1D array that stores only useful matrix values.
Index formula	For diagonal positions, use `A[i - 1]`.
Class	A C++ blueprint that stores data and functions together.
Constructor	A special function that runs when an object is created.
Destructor	A special function that runs when an object is destroyed.
Dynamic memory	Memory allocated during program execution using `new`.

3. Lesson 3 vs Lesson 4

In Lesson 3, we created a C++ class for a diagonal matrix.

The class used:

```
int n;
int *A;
```

It also had:

```
Diagonal(int n);
~Diagonal();
set();
get();
display();
```

Lesson 4 makes the program more complete.

The main difference is that Lesson 4 adds a **default constructor**.

So now the class has:

```
Diagonal();      // default constructor
Diagonal(int n); // parameterized constructor
~Diagonal();     // destructor
```

This means we can create a diagonal matrix object in two ways:

```
Diagonal d1;     // uses default constructor
Diagonal d2(4);  // uses parameterized constructor
```

4. What is a complete C++ diagonal matrix program?

A complete C++ diagonal matrix program should contain:

1. A class named `Diagonal`
2. Private data members
3. Constructors

4. Destructor
5. Methods to set, get, and display values
6. A `main()` function to test the class

The class should be able to:

- Create a diagonal matrix
- Store diagonal values
- Ignore non-diagonal values
- Return stored diagonal values
- Return zero for non-diagonal positions
- Display the full square matrix
- Release dynamic memory when the object is destroyed

---

5. Class structure

The class structure is:

```
class Diagonal {
private:
    int *A;
    int n;

public:
    Diagonal();
    Diagonal(int n);
    ~Diagonal();

    void Set(int i, int j, int x);
    int Get(int i, int j);
    void Display();
};
```

This class has two main sections:

```
private:
```

and:

```
public:
```

---

6. Private data members

The private data members are:

```
int *A;
int n;
```

Meaning of `int *A`

```
int *A;
```

means `A` is a pointer to an integer array.

This array stores only the diagonal values.

Example:

```
A = [5, 8, 9, 12]
```

This represents the full matrix:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

Only the diagonal values are stored.

The zeros are not stored.

Meaning of `int n`

```
int n;
```

stores the dimension of the matrix.

If:

```
n = 4;
```

then the matrix is treated as a `4 × 4` matrix.

Even though we store only 4 diagonal values, the visible matrix has:

```
4 × 4 = 16 positions
```

---

7. Why are `A` and `n` private?

The data members are private so outside code cannot directly change them.

This protects the diagonal matrix rule.

For example, outside code should not directly do this:

```
d.A[2] = 99;
```

Instead, the user should call:

```
d.Set(3, 3, 99);
```

The `Set()` method checks whether the position is diagonal before storing the value.

So private data helps the class protect itself.

The object controls how its data changes.

---

8. Public methods

The public methods are:

```
Diagonal();
Diagonal(int n);
~Diagonal();
void Set(int i, int j, int x);
int Get(int i, int j);
void Display();
```

These are the functions that outside code is allowed to use.

Method	Purpose
`Diagonal()`	Creates a default matrix.
`Diagonal(int n)`	Creates a matrix with a user-provided size.
`~Diagonal()`	Releases dynamic memory.
`Set()`	Stores a value if the position is diagonal.
`Get()`	Returns a stored value or zero.
`Display()`	Prints the full matrix.

---

9. Default constructor

A default constructor is a constructor that does not take any arguments.

It looks like this:

```
Diagonal() {
    n = 2;
    A = new int[2];
}
```

This constructor runs when we create an object like this:

```
Diagonal d;
```

No size is given.

So the default constructor gives the object a safe default size.

In this lesson, the default size is:

```
2 × 2
```

So internally:

```
n = 2
A has 2 integer spaces
```

The visible matrix is:

```
? 0
0 ?
```

The `?` values are diagonal values that can be set later.

---

10. Why do we need a default constructor?

The default constructor allows this:

```
Diagonal d;
```

Without a default constructor, this line would not work if the class only had:

```
Diagonal(int n);
```

The default constructor makes sure the object is still valid even when no size is provided.

A valid object should have:

```
A matrix size
A memory array
```

So this code:

```
Diagonal() {
    n = 2;
    A = new int[2];
}
```

creates a small valid diagonal matrix.

---

11. Important correction about `new int[2]`

This code:

```
A = new int[2];
```

allocates memory for 2 integers.

But it does **not always initialize the values to zero**.

That means the array values may contain garbage values until we set them.

A safer version is:

```
A = new int[2]{};
```

The `{}` initializes the values to zero.

So this:

```
A = new int[2]{};
```

creates:

```
A = [0, 0]
```

This is safer for beginners.

---

12. Parameterized constructor

A parameterized constructor receives a value.

In this lesson, it receives the matrix size.

```
Diagonal(int n) {
    this->n = n;
    A = new int[n];
}
```

This constructor runs when we create an object like this:

```
Diagonal d(4);
```

This means:

```
Create a diagonal matrix of size 4 × 4.
```

Since it is diagonal, we only need to store 4 values.

So internally:

```
n = 4
A has 4 integer spaces
```

The full matrix has 16 visible positions, but the array stores only 4 values.

---

13. Understanding `this->n = n`

The parameterized constructor has this line:

```
this->n = n;
```

There are two different `n` names here.

The first one is:

```
this->n
```

This means the object's own `n` data member.

The second one is:

```
n
```

This means the constructor parameter.

So:

```
this->n = n;
```

means:

```
Store the constructor parameter into the object's n variable.
```

Example:

```
Diagonal d(4);
```

The constructor receives:

```
n = 4
```

Then:

```
this->n = n;
```

stores 4 into the object's private `n`.

---

14. Safer parameterized constructor

The basic version is:

```
Diagonal(int n) {
    this->n = n;
    A = new int[n];
}
```

A safer beginner-friendly version is:

```
Diagonal(int n) {
    this->n = n;
    A = new int[n]{};
}
```

The `{}` initializes all values to zero.

Example:

```
Diagonal d(4);
```

creates:

```
A = [0, 0, 0, 0]
```

Then later we can set values:

```
d.Set(1, 1, 5);
d.Set(2, 2, 8);
d.Set(3, 3, 9);
d.Set(4, 4, 12);
```

Then the array becomes:

```
A = [5, 8, 9, 12]
```

---

15. Destructor

A destructor is a special function that runs automatically when an object is destroyed.

The destructor is:

```
~Diagonal() {
    delete[] A;
}
```

The destructor releases the memory that was created using `new`.

In the constructor, we wrote:

```
A = new int[n];
```

So in the destructor, we must write:

```
delete[] A;
```

---

16. Why do we need a destructor?

We need a destructor because the class uses dynamic memory.

Dynamic memory is created using:

```
new int[n]
```

The program asks the heap for memory.

When the object is finished, the program should return that memory.

If we do not release the memory, we may create a memory leak.

A memory leak means:

```
The program asked for memory but never gave it back.
```

So this rule is important:

```
new[] must be matched with delete[]
```

Correct:

```
A = new int[n];
delete[] A;
```

Wrong:

```
A = new int[n];
delete A;
```

Why is `delete A` wrong?

Because `A` points to an array, not one single integer.

For arrays, use:

```
delete[] A;
```

---

17. `Set()` method

The `Set()` method stores a value in the matrix.

Code:

```
void Set(int i, int j, int x) {
    if (i == j) {
        A[i - 1] = x;
    }
}
```

Meaning:

- `i` is the row number.
- `j` is the column number.
- `x` is the value to store.

The method stores `x` only if:

```
i == j
```

If `i != j`, the method does nothing.

Why?

Because non-diagonal positions must stay zero in a diagonal matrix.

---

18. Why `Set()` uses `A[i - 1]`

Matrix positions are 1-based in `Set()`.

That means positions start from 1:

```
M[1,1]
M[2,2]
M[3,3]
M[4,4]
```

But C++ arrays are 0-based:

```
A[0]
A[1]
A[2]
A[3]
```

So we subtract 1.

Mapping:

Matrix position	Array position
`M[1,1]`	`A[0]`
`M[2,2]`	`A[1]`
`M[3,3]`	`A[2]`
`M[4,4]`	`A[3]`

Formula:

```
A[i - 1]
```

This formula is used only when:

```
i == j
```

---

19. Dry run of `Set()` for a diagonal position

Call:

```
d.Set(3, 3, 9);
```

Step 1: Identify the values.

```
i = 3
j = 3
x = 9
```

Step 2: Check the diagonal condition.

```
i == j
3 == 3
true
```

Step 3: Calculate the array index.

```
index = i - 1
index = 3 - 1
index = 2
```

Step 4: Store the value.

```
A[2] = 9;
```

So `9` is stored as the third diagonal value.

---

20. Dry run of `Set()` for a non-diagonal position

Call:

```
d.Set(2, 4, 99);
```

Step 1: Identify the values.

```
i = 2
j = 4
x = 99
```

Step 2: Check the diagonal condition.

```
i == j
2 == 4
false
```

Step 3: Do nothing.

The value `99` is ignored.

Why?

Because `(2,4)` is not on the main diagonal.

In a diagonal matrix, `(2,4)` must be zero.

---

21. `Get()` method

The `Get()` method returns the value at a matrix position.

Code:

```
int Get(int i, int j) {
    if (i == j) {
        return A[i - 1];
    }
    return 0;
}
```

Meaning:

- If the position is diagonal, return the stored value.
- If the position is not diagonal, return zero.

So:

```
d.Get(2, 2);
```

returns a stored value.

But:

```
d.Get(1, 4);
```

returns `0`.

---

22. Why `Get()` returns zero for non-diagonal positions

In a diagonal matrix, every non-diagonal position is a structural zero.

Examples:

```
M[1,2] = 0
M[1,4] = 0
M[3,1] = 0
M[4,2] = 0
```

These zeros are not stored in the array.

The program simply knows they must be zero because of the diagonal matrix rule.

So if:

```
i != j
```

then `Get()` returns:

```
return 0;
```

without checking the array.

---

23. Dry run of `Get()` for a diagonal position

Assume:

```
A = [5, 8, 9, 12]
```

Call:

```
d.Get(2, 2);
```

Step 1: Check the condition.

```
2 == 2
true
```

Step 2: Calculate the index.

```
index = i - 1
index = 2 - 1
index = 1
```

Step 3: Return the stored value.

```
return A[1];
```

Since:

```
A[1] = 8
```

The answer is:

```
8
```

---

24. Dry run of `Get()` for a non-diagonal position

Call:

```
d.Get(1, 4);
```

Step 1: Check the condition.

```
1 == 4
false
```

Step 2: Return zero.

```
return 0;
```

The method does not check the array.

It already knows `(1,4)` is not on the diagonal.

So the value must be zero.

---

25. `Display()` method

The `Display()` method prints the full visible matrix.

Code:

```
void Display() {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) {
                cout << A[i] << " ";
            } else {
                cout << "0 ";
            }
        }
        cout << endl;
    }
}
```

The method uses two loops:

- Outer loop: controls rows
- Inner loop: controls columns

For each position:

- If `i == j`, print the stored diagonal value.
- If `i != j`, print `0`.

---

26. Why `Display()` uses nested loops

The stored array contains only diagonal values.

Example:

```
A = [5, 8, 9, 12]
```

But the displayed matrix must look like a full `4 × 4` matrix:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

To print the full matrix, the program must visit every row and every column.

For a `4 × 4` matrix, it prints:

```
4 × 4 = 16 positions
```

That is why `Display()` uses nested loops.

---

27. Important indexing difference in Lesson 4

This is one of the most important ideas in Lesson 4.

`Set()` and `Get()` use **1-based matrix positions**.

Example:

```
d.Set(1, 1, 5);
d.Set(2, 2, 8);
d.Set(3, 3, 9);
d.Set(4, 4, 12);
```

Here, the matrix positions start at 1.

So the formula is:

```
A[i - 1]
```

But `Display()` uses **0-based loops**.

Example:

```
for (int i = 0; i < n; i++)
```

Here, `i` starts at 0.

So when `i = 0`, the correct array position is:

```
A[0]
```

That is why `Display()` uses:

```
A[i]
```

not:

```
A[i - 1]
```

---

28. Why `Display()` uses `A[i]` instead of `A[i - 1]`

In `Display()`, the loop starts from 0:

```
for (int i = 0; i < n; i++)
```

So the values of `i` are:

```
0, 1, 2, 3
```

for a `4 × 4` matrix.

These are already valid array indexes:

```
A[0], A[1], A[2], A[3]
```

So the display method uses:

```
A[i]
```

If we used:

```
A[i - 1]
```

then when `i = 0`, we would get:

```
A[-1]
```

That is invalid.

So the rule is:

```
If the loop starts at 1, use A[i - 1].
If the loop starts at 0, use A[i].
```

---

29. 1-based vs 0-based summary

Situation	Loop or input starts at	Correct formula
`Set(i, j, x)`	1	`A[i - 1]`
`Get(i, j)`	1	`A[i - 1]`
`Display()` in Lesson 4	0	`A[i]`

This is why formulas must match the indexing style being used.

---

30. Full Lesson 4 program

```
#include <iostream>
using namespace std;
```

```
class Diagonal {
private:
    int *A;
    int n;

public:
    Diagonal() {
        n = 2;
        A = new int[2];
    }

    Diagonal(int n) {
        this->n = n;
        A = new int[n];
    }

    ~Diagonal() {
        delete[] A;
    }

    void Set(int i, int j, int x) {
        if (i == j) {
            A[i - 1] = x;
        }
    }

    int Get(int i, int j) {
        if (i == j) {
            return A[i - 1];
        }
        return 0;
    }

    void Display() {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (i == j) {
                    cout << A[i] << " ";
                } else {
                    cout << "0 ";
                }
            }
            cout << endl;
        }
    }
};

int main() {
```

```
    Diagonal d(4);

    d.Set(1, 1, 5);
    d.Set(2, 2, 8);
    d.Set(3, 3, 9);
    d.Set(4, 4, 12);

    d.Display();

    return 0;
}
```

---

31. Safer beginner-friendly version of the program

This version initializes the dynamic arrays to zero.

```
#include <iostream>
using namespace std;

class Diagonal {
private:
    int *A;
    int n;

public:
    Diagonal() {
        n = 2;
        A = new int[2]{};
    }

    Diagonal(int n) {
        this->n = n;
        A = new int[n]{};
    }

    ~Diagonal() {
        delete[] A;
    }

    void Set(int i, int j, int x) {
        if (i == j) {
            A[i - 1] = x;
        }
    }
```

```
    int Get(int i, int j) {
        if (i == j) {
            return A[i - 1];
        }
        return 0;
    }

    void Display() {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (i == j) {
                    cout << A[i] << " ";
                } else {
                    cout << "0 ";
                }
            }
            cout << endl;
        }
    }
};

int main() {
    Diagonal d(4);

    d.Set(1, 1, 5);
    d.Set(2, 2, 8);
    d.Set(3, 3, 9);
    d.Set(4, 4, 12);

    d.Display();

    return 0;
}
```

The safer version uses:

```
new int[n]{}
```

instead of:

```
new int[n]
```

This helps avoid garbage values before the user stores real values.

---

32. Explanation of the full program

Header file

```
#include <iostream>
```

This is needed because the program uses:

```
cout
endl
```

Namespace

```
using namespace std;
```

This allows us to write:

```
cout
endl
```

instead of:

```
std::cout
std::endl
```

Class name

```
class Diagonal
```

This creates a class named `Diagonal`.

The class represents a diagonal matrix.

Private section

```
private:
    int *A;
    int n;
```

This stores:

```
A = dynamic array of diagonal values
n = matrix dimension
```

Public section

```
public:
```

This contains the functions that outside code can call.

Default constructor

```
Diagonal() {
    n = 2;
    A = new int[2];
}
```

This creates a default `2 × 2` diagonal matrix.

Parameterized constructor

```
Diagonal(int n) {
    this->n = n;
    A = new int[n];
}
```

This creates a diagonal matrix of size `n × n`.

Destructor

```
~Diagonal() {
    delete[] A;
}
```

This releases the dynamic array.

`Set()` method

```
void Set(int i, int j, int x)
```

This stores `x` only if `i == j`.

`Get()` method

```
int Get(int i, int j)
```

This returns:

- `A[i - 1]` if `i == j`
- `0` if `i != j`

`Display()` method

```
void Display()
```

This prints the full matrix using:

- Stored diagonal values
- Fake zeros for non-diagonal positions

`main()` function

```
int main() {
    Diagonal d(4);

    d.Set(1, 1, 5);
    d.Set(2, 2, 8);
    d.Set(3, 3, 9);
    d.Set(4, 4, 12);

    d.Display();

    return 0;
}
```

This creates a `4 × 4` diagonal matrix, stores four diagonal values, and displays the matrix.

---

33. Main function trace

The `main()` function starts here:

```
int main() {
```

Then this line runs:

```
Diagonal d(4);
```

This calls the parameterized constructor.

The object becomes:

```
n = 4
A has 4 spaces
```

If using the safer version:

```
A = new int[4]{};
```

then initially:

```
A = [0, 0, 0, 0]
```

---

34. Trace of setting values

First call

```
d.Set(1, 1, 5);
```

Check:

```
1 == 1 → true
```

Index:

```
1 - 1 = 0
```

Store:

```
A[0] = 5
```

Array now:

```
[5, 0, 0, 0]
```

Second call

```
d.Set(2, 2, 8);
```

Check:

```
2 == 2 → true
```

Index:

```
2 - 1 = 1
```

Store:

```
A[1] = 8
```

Array now:

```
[5, 8, 0, 0]
```

Third call

```
d.Set(3, 3, 9);
```

Check:

```
3 == 3 → true
```

Index:

```
3 - 1 = 2
```

Store:

```
A[2] = 9
```

Array now:

```
[5, 8, 9, 0]
```

Fourth call

```
d.Set(4, 4, 12);
```

Check:

```
4 == 4 → true
```

Index:

```
4 - 1 = 3
```

Store:

```
A[3] = 12
```

Array now:

```
[5, 8, 9, 12]
```

---

35. Trace of `Display()`

The display function uses:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (i == j) {
            cout << A[i] << " ";
        } else {
            cout << "0 ";
        }
    }
    cout << endl;
}
```

For `n = 4`, the loop values are:

```
i = 0, 1, 2, 3
j = 0, 1, 2, 3
```

Stored array:

```
A = [5, 8, 9, 12]
```

Row 1

When `i = 0`:

```
j = 0: i == j, print A[0] = 5
j = 1: i != j, print 0
j = 2: i != j, print 0
j = 3: i != j, print 0
```

Output row:

```
5 0 0 0
```

Row 2

When `i = 1`:

```
j = 0: i != j, print 0
j = 1: i == j, print A[1] = 8
```

```
j = 2: i != j, print 0
j = 3: i != j, print 0
```

Output row:

```
0 8 0 0
```

Row 3

When `i = 2`:

```
j = 0: i != j, print 0
j = 1: i != j, print 0
j = 2: i == j, print A[2] = 9
j = 3: i != j, print 0
```

Output row:

```
0 0 9 0
```

Row 4

When `i = 3`:

```
j = 0: i != j, print 0
j = 1: i != j, print 0
j = 2: i != j, print 0
j = 3: i == j, print A[3] = 12
```

Output row:

```
0 0 0 12
```

---

36. Expected output

The expected output is:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

This is the full visible matrix.

But the actual stored array is only:

```
A = [5, 8, 9, 12]
```

The zeros are printed by the program.

They are not stored.

---

37. Stored array vs visible matrix

The stored array is what the computer actually keeps in memory.

```
A = [5, 8, 9, 12]
```

The visible matrix is what the user sees.

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

The important difference:

```
Stored form: only diagonal values
Visible form: full square matrix
```

This is the main memory-saving idea.

---

38. Why storage is efficient

A normal `4 × 4` matrix stores every value.

That means:

```
4 × 4 = 16 values
```

But a diagonal matrix stores only the diagonal values.

That means:

```
4 values
```

For a general `N × N` matrix:

Matrix type	Stored values
Normal matrix	N^2
Diagonal matrix	N

So diagonal matrix storage is much smaller.

Example for $N = 1000$:

Matrix type	Stored values
Normal matrix	$1,000,000$
Diagonal matrix	$1,000$

That is a big memory saving.

---

39. Time and space complexity

Constructor complexity

The constructor allocates an array of size N.

```
A = new int[n];
```

So space complexity is:

```
O(N)
```

From this lesson's simplified view, the constructor time is treated as:

```
O(1)
```

because it mainly performs assignment and allocation.

`Set()` complexity

The `Set()` method does:

- One condition check
- One index calculation if needed
- One assignment if needed

It does not use loops.

So time complexity is:

```
O(1)
```

`Get()` complexity

The `Get()` method does:

- One condition check
- One array access if needed
- Or immediately returns zero

It does not use loops.

So time complexity is:

```
O(1)
```

`Display()` complexity

The `Display()` method uses two nested loops.

For an `N × N` matrix, it prints:

```
N² positions
```

So time complexity is:

```
O(N²)
```

Even though storage is `O(N)`, display is still `O(N²)` because the full visible matrix has `N²` positions.

---

40. Complexity table

Operation	Time Complexity	Space Complexity	Reason
Matrix storage	—	`O(N)`	Store only `N` diagonal values.
Constructor	`O(1)`	`O(N)`	Allocates one array of size `N`.
`Set()`	`O(1)`	—	One condition and one index expression.
`Get()`	`O(1)`	—	One condition and one possible array lookup.
`Display()`	`O(N²)`	—	Prints every visible matrix position.

---

41. Common beginner mistakes

Mistake 1: Using `A[i - 1]` in 0-based `Display()`

Wrong:

```
for (int i = 0; i < n; i++) {
    cout << A[i - 1];
}
```

When `i = 0`, this becomes:

```
A[-1]
```

That is invalid.

Correct:

```
cout << A[i];
```

because `i` already starts at 0.

Mistake 2: Forgetting the destructor

Wrong:

```
~Diagonal() {
}
```

Correct:

```
~Diagonal() {
    delete[] A;
}
```

Because the constructor used:

```
new int[n]
```

Mistake 3: Using `delete` instead of `delete[]`

Wrong:

```
delete A;
```

Correct:

```
delete[] A;
```

Because `A` points to an array.

Mistake 4: Thinking all displayed zeros are stored

The output may look like this:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

But the stored array is only:

```
[5, 8, 9, 12]
```

The zeros are printed logically.

They are not stored.

Mistake 5: Trying to store non-diagonal values

Wrong idea:

```
d.Set(1, 3, 99);
```

should store `99` somewhere.

Correct idea:

```
d.Set(1, 3, 99);
```

should do nothing.

Reason:

```
1 != 3
```

So `(1,3)` is not diagonal.

It must stay zero.

Mistake 6: Forgetting that `Set()` and `Get()` use 1-based positions

For `Set()` and `Get()`, this is correct:

```
d.Set(1, 1, 5); // stores in A[0]
```

This mapping is used:

```
M[1,1] → A[0]
M[2,2] → A[1]
M[3,3] → A[2]
```

So the formula is:

```
A[i - 1]
```

---

42. Beginner mental model

Think of the class as a small machine.

Inside the machine:

```
n = matrix size
A = stored diagonal values
```

Outside the machine, the user can press buttons:

```
Set()
Get()
Display()
```

The user does not directly touch the stored array.

The class protects the diagonal matrix rule.

When the user calls:

```
d.Set(i, j, x);
```

The class checks:

```
if (i == j)
```

If true, it stores the value.

If false, it ignores the value.

When the user calls:

```
d.Get(i, j);
```

The class checks:

```
if (i == j)
```

If true, it returns a stored value.

If false, it returns zero.

When the user calls:

```
d.Display();
```

The class prints a full matrix using stored values and fake zeros.

---

43. Practice example 1

Suppose:

```
Diagonal d(3);
```

Then:

```
d.Set(1, 1, 10);
d.Set(2, 2, 20);
d.Set(3, 3, 30);
```

Stored array:

```
A = [10, 20, 30]
```

Visible matrix:

```
10 0 0
0 20 0
0 0 30
```

Questions:

```
d.Get(1, 1)
d.Get(2, 2)
d.Get(3, 1)
d.Get(1, 3)
```

Answers:

Operation	Answer	Reason
`d.Get(1, 1)`	`10`	Diagonal position, return `A[0]`.
`d.Get(2, 2)`	`20`	Diagonal position, return `A[1]`.
`d.Get(3, 1)`	`0`	Non-diagonal position.
`d.Get(1, 3)`	`0`	Non-diagonal position.

---

44. Practice example 2

Suppose:

```
Diagonal d(4);
```

Then:

```
d.Set(1, 1, 4);
d.Set(2, 2, 6);
d.Set(3, 3, 8);
d.Set(4, 4, 10);
```

Stored array:

```
A = [4, 6, 8, 10]
```

Visible matrix:

```
4 0 0 0
0 6 0 0
0 0 8 0
0 0 0 10
```

Questions and answers:

Operation	Answer	Reason
`d.Get(4, 4)`	`10`	Diagonal, return `A[3]`.
`d.Get(2, 4)`	`0`	Non-diagonal.
`d.Set(3, 3, 99)`	Stores `99` at `A[2]`	Diagonal.
`d.Set(1, 4, 77)`	Ignored	Non-diagonal.

After:

```
d.Set(3, 3, 99);
```

The array becomes:

```
A = [4, 6, 99, 10]
```

The visible matrix becomes:

```
4 0 0 0
0 6 0 0
0 0 99 0
0 0 0 10
```

---

45. Key rules from Lesson 4

Diagonal condition

```
i == j
```

This means the position is on the main diagonal.

Non-diagonal condition

```
i != j
```

This means the position is outside the main diagonal.

`Set()` rule

```
If i == j, store x in A[i - 1].
If i != j, do nothing.
```

`Get()` rule

```
If i == j, return A[i - 1].
If i != j, return 0.
```

`Display()` rule

```
Loop through all visible positions.
Print A[i] when i == j.
Print 0 when i != j.
```

Memory rule

```
new[] must be matched with delete[]
```

Indexing rule

```
Set/Get use 1-based matrix positions, so use A[i - 1].
Display uses 0-based loops, so use A[i].
```

46. Lesson 4 final summary

Lesson 4 teaches how to build a complete C++ diagonal matrix program.

The matrix type is still a diagonal matrix.

The rule is still:

```
i == j
```

Only diagonal values are stored.

All non-diagonal values are structural zeros.

The class stores:

```
int *A;
int n;
```

`A` stores the diagonal values.

`n` stores the matrix dimension.

The class includes:

```
Diagonal();
Diagonal(int n);
~Diagonal();
Set();
Get();
Display();
```

The default constructor creates a safe default `2 × 2` matrix.

The parameterized constructor creates a matrix of the requested size.

The destructor releases the dynamic array using:

```
delete[] A;
```

The `Set()` method stores values only when `i == j`.

The `Get()` method returns stored values for diagonal positions and returns `0` for non-diagonal positions.

The `Display()` method prints the full matrix using nested loops.

The most important new indexing idea is:

```
Set/Get use 1-based positions, so they use A[i - 1].
Display uses 0-based loops, so it uses A[i].
```

Storage is efficient because a diagonal matrix stores only `N` values instead of `N²` values.

So storage complexity is:

```
O(N)
```

`Set()` and `Get()` are:

```
O(1)
```

because they use direct formulas.

`Display()` is:

```
O(N²)
```

because it prints every visible position in the full matrix.

---

47. Exit questions

You should be able to answer these before moving to Lesson 5.

1. What is the purpose of Lesson 4?
2. What is a default constructor?
3. What does the default constructor do in this lesson?
4. What is a parameterized constructor?
5. What does `this->n = n` mean?
6. Why do we use `new int[n]`?

7. Why is `new int[n]{}` safer for beginners?
8. What does the destructor do?
9. Why do we use `delete[] A` instead of `delete A`?
10. What does `Set(3, 3, 9)` do?
11. What does `Set(2, 4, 99)` do?
12. What does `Get(2, 2)` return if `A[1] = 8`?
13. Why does `Get(1, 4)` return zero?
14. Why does `Display()` use nested loops?
15. Why does `Display()` use `A[i]` instead of `A[i - 1]`?
16. What is the storage complexity of a diagonal matrix?
17. What is the time complexity of `Set()`?
18. What is the time complexity of `Get()`?
19. What is the time complexity of `Display()`?
20. What is the main indexing lesson from Lesson 4?

---

48. Short answers to exit questions

1. Lesson 4 teaches a complete C++ diagonal matrix class program.
2. A default constructor creates an object without receiving arguments.
3. It creates a default `2 × 2` diagonal matrix.
4. A parameterized constructor creates an object using a given value, such as matrix size.
5. It stores the constructor parameter `n` into the object's private `n` variable.
6. We use `new int[n]` to create a dynamic array of the correct size.
7. `new int[n]{}` initializes the array values to zero.
8. The destructor releases the dynamic memory.
9. We use `delete[] A` because `A` points to an array.
10. It stores `9` at `A[2]`.
11. It does nothing because `(2,4)` is not diagonal.
12. It returns `8`.
13. Because `(1,4)` is a structural zero.
14. Because it must print every visible matrix position.
15. Because the display loop starts from `0`, so `i` is already a valid array index.
16. Storage complexity is `O(N)`.
17. `Set()` is `O(1)`.
18. `Get()` is `O(1)`.
19. `Display()` is `O(N²)`.
20. The main indexing lesson is: use `A[i - 1]` for 1-based matrix positions, but use `A[i]` for 0-based loops.